



შპს 5

Red-Black Tree

Iakob Chaladze

დავალების აღწერა

თავი 1 — Node.h.....	2
თავი 2 — RedBlackTree.h	4
თავი 3 — RedBlackTree.cpp.....	6
თავი 4 — Driver.cpp.....	26
ტესტირების შედეგები.....	28

თავი 1 — Node.h

```
1 // Node.h
2 #pragma once
3
4 // Forward declaration to allow RedBlackTree to access Node's private members
5 class RedBlackTree;
6
7 // Node structure used in Red-Black Tree
8 class Node
9 {
10     // RedBlackTree needs direct access to private members
11     friend class RedBlackTree;
12
13 private:
14     int value; // Value stored in the node
15     bool isBlack; // Node color: true = black, false = red
16     bool isLeftChild; // Indicates whether this node is a left child of its parent
17
18     Node* parent; // Pointer to parent node
19     Node* left; // Pointer to left child
20     Node* right; // Pointer to right child
21
22 public:
23     // Default Constructor
24     Node() : value(0), isBlack(false), isLeftChild(false), parent(nullptr), left(nullptr), right(nullptr) {}
25     // Constructor for creating regular tree nodes
26     Node(int val, bool black, bool isLeftChild, Node* p, Node* l, Node* r)
27     : value(val), isBlack(black), isLeftChild(isLeftChild), parent(p), left(l), right(r) {}
28 };
```

ზოგადი დანიშნულება

Node.h ფაილი განსაზღვრავს წითელ-შავი ხის საბაზო სამშენებლო ბლოკს — კლასს **Node**. ეს ფაილი ახდენს მონაცემთა ინკაფსულაციას, რომლებიც აუცილებელია ხის სტრუქტურის აგებისთვის. კლასი **RedBlackTree** გამოცხადებულია როგორც **friend**, რაც მას აძლევს პირდაპირ წვდომას **Node**-ის პრივატულ წევრებზე, რაც ამარტივებს ბალანსირების ალგორითმების იმპლემენტაციას.

კლასი Node

ველები (Private)

- **value (int):** ინახავს კვანძის მნიშვნელობას.
- **isBlack (bool):** განსაზღვრავს კვანძის ფერს. true ნიშნავს შავს, ხოლო false — წითელს.
- **isLeftChild (bool):** ეს არის სპეციფიკური ველი ამ იმპლემენტაციისთვის. ის ინახავს ინფორმაციას, არის თუ არა მოცემული კვანძი მშობლის მარცხენა შვილი. ეს ამარტივებს ლოგიკას როტაციების დროს, რადგან არ გვჭირდება მშობლის შვილების შედარება იმის გასაგებად, თუ რომელ მხარეს ვართ.
- ***parent, left, right (Node)** - მიმთითებლები მშობელზე, მარცხენა და მარჯვენა შვილებზე.

კონსტრუქტორები

- **Node() (Default):** ახდენს კვანძის ინიციალიზაციას ნულოვანი მნიშვნელობებით
- **Node(int val, ...) (Parameterized):** გამოიყენება ახალი კვანძის შექმნისას კონკრეტული მნიშვნელობებით, ფერით და მშობლის/შვილების მიმთითებლებით.

თავი 2 — RedBlackTree.h

```
1 // RedBlackTree.h
2 #pragma once
3 #include <iostream>
4 #include "Node.h"
5
6 class RedBlackTree
7 {
8     Node* Root; // Root of the tree
9     Node* nullLeaf; // Node representing all null leaves
10    std::size_t nodeCount; // Number of elements in the tree
11
12 private:
13     void NullLeaf(); // Creates and initializes the null leaf
14     Node* SearchRecHelper(Node* node, int value) const; // Recursive search helper
15     Node* SearchIterHelper(int value) const; // Iterative search helper
16
17     // Recursive BST insertion helper
18     Node* InsertBST(Node* currentRoot, Node* parent, int value, Node*& insertedNode, bool isLeftChild);
19     void LeftRotate(Node* rotationRoot); // Left rotation around a given node
20     void RightRotate(Node* rotationRoot); // Right rotation around a given node
21     void ColorFlip(Node* node); // Flips colors of node and its children (used during rebalancing)
22     void FixInsert(Node* insertedNode); // Restores Red-Black properties after insertion
23     void FixDelete(Node* movedUpNode); // Restores Red-Black properties after insertion
24     void Transplant(Node* oldSubtreeRoot, Node* newSubtreeRoot); // Replaces one subtree with another
25
26     Node* Minimum(Node* node) const; // Finds the minimum node in a subtree
27     // Copies subtree recursively (used in copy constructor / assignment operator)
28     Node* CopySubtree(const Node* otherNode, const Node* otherNullLeaf, Node* newParent);
29     void ClearSubtree(Node* node); // Deletes all nodes in a subtree
30
31     // Traversal helpers
32     void DisplayInOrder(std::ostream& out, Node* node) const;
33     void DisplayPreOrder(std::ostream& out, Node* node) const;
34     void DisplayPostOrder(std::ostream& out, Node* node) const;
35     // Deletes a node given its pointer
36     void RemoveByNode(Node* nodeToDelete);
37
38 public:
39     // Constructors / Destructor
40     RedBlackTree();
41     RedBlackTree(const RedBlackTree& other);
42     RedBlackTree& operator = (const RedBlackTree& other);
43     ~RedBlackTree();
44     // Tree state queries
45     std::size_t getSize() const;
46     bool isEmpty() const;
47     // Insert operations
48     void InsertIter(int value);
49     void InsertRec(int value);
50     // Remove operations
51     void RemoveIter(int value);
52     void RemoveRec(int value);
53     // Search operations
54     bool SearchIter(int value) const;
55     bool SearchRec(int value) const;
56     // Tree traversals
57     void DisplayInOrder(std::ostream& out = std::cout) const;
58     void DisplayPreOrder(std::ostream& out = std::cout) const;
59     void DisplayPostOrder(std::ostream& out = std::cout) const;
60 };
```

ზოგადი დანიშნულება

RedBlackTree.h ფაილი შეიცავს **RedBlackTree** კლასის ინტერფეისს. აქ ხდება ფუნქციების პროტოტიპების დეკლარაცია, ხის მდგომარეობის ცვლადების განსაზღვრა და წვდომის დონეების (public/private) გამოიყვნა.

კლასი RedBlackTree

მონაცემები (Private)

- **Root:** მიმთითებელი ხის ფესვზე.
- **nullLeaf:** სპეციალური კვანძი. სტანდარტული nullptr-ის ნაცვლად, ფოთლები მიუთითებენ ამ ობიექტზე. ეს კვანძი ყოველთვის შავია, რაც ამარტივებს FixInsert და FixDelete ალგორითმებს (აღარ არის საჭირო nullptr შემოწმებები ფერის გაგებისას).
- **nodeCount:** ხეში არსებული ელემენტების რაოდენობა.

დამხმარე ფუნქციები (Private)

ეს მეთოდები უზრუნველყოფენ ხის შიდა ლოგიკას:

- **ბალანსირება:** LeftRotate, RightRotate, ColorFlip, FixInsert, FixDelete.
- **სტრუქტურული ცვლილებები:** Transplant (ქვეხის ჩანაცვლება), InsertBST (რეკურსიული ჩამატება).
- **მეხსიერების მართვა:** CopySubtree (ღრმა ასლი), ClearSubtree (მეხსიერების გასუფთავება), NullLeaf (სენტინელის ინიციალიზაცია).
- **ძებნა და შემოვლა:** SearchRecHelper, SearchIterHelper, DisplayInOrder და ა.შ.

საჯარო ინტერფეისი (Public)

- **კონსტრუქტორები:** Default, Copy Constructor და Assignment Operator (operator=) უზრუნველყოფენ ობიექტის სწორ კოპირებას.
- **დესტრუქტორი:** ~RedBlackTree() ასუფთავებს მეხსიერებას.
- **ოპერაციები:** InsertIter, InsertRec, RemoveIter, RemoveRec, SearchIter, SearchRec — მომხმარებელს შეუძლია აირჩიოს იტერაციული ან რეკურსიული მიდგომა.

თავი 3 — RedBlackTree.cpp

```
1  #include "RedBlackTree.h"
2
3  // Returns the number of nodes in the tree
4  std::size_t RedBlackTree::getSize() const { return nodeCount; }
5  // Checks if the tree is empty
6  bool RedBlackTree::isEmpty() const { return nodeCount == 0; }
7
8  // This node represents all Null leaves in the Red-Black Tree
9  void RedBlackTree::NullLeaf()
10 {
11     nullLeaf = new Node(); // Create new node for nullLeaf
12     nullLeaf->isBlack = true; // Null leaves are always black
13     nullLeaf->isLeftChild = false;
14     // Set all pointers to point to itself (prevents nullptr issues)
15     nullLeaf->parent = nullLeaf->left = nullLeaf->right = nullLeaf;
16 }
17
18 // Recursively copies a subtree from another tree
19 // Returns pointer to the root of the copied subtree
20 Node* RedBlackTree::CopySubtree(const Node* otherNode, const Node* otherNullLeaf, Node* newParent)
21 {
22     // Base case: reached null leaf in source tree
23     if (otherNode == otherNullLeaf) return nullLeaf;
24
25     // Create new node with same value and color
26     Node* newNode = new Node(otherNode->value, otherNode->isBlack,
27                             false, newParent, nullLeaf, nullLeaf);
28     // Recursively copy left subtree
29     newNode->left = CopySubtree(otherNode->left, otherNullLeaf, newNode);
30     if (newNode->left != nullLeaf)
31     {
32         newNode->left->isLeftChild = true; // Set left child flag
33     }
34     // Recursively copy right subtree
35     newNode->right = CopySubtree(otherNode->right, otherNullLeaf, newNode);
36     if (newNode->right != nullLeaf)
37     {
38         newNode->right->isLeftChild = false; // Set right child flag
39     }
40     return newNode;
41 }
42
43 // Recursively deletes all nodes in a subtree
44 void RedBlackTree::ClearSubtree(Node* node)
45 {
46     // Base case: reached null leaf
47     if (node == nullLeaf) return;
48
49     // Post-order traversal: delete children first, then parent
50     ClearSubtree(node->left);
51     ClearSubtree(node->right);
52     delete node; // Delete current node
53 }
54
55 // Recursive helper function for searching
56 // Returns pointer to found node, or nullLeaf if not found
57 Node* RedBlackTree::SearchRecHelper(Node* node, int value) const
58 {
59     // Base case: value not found
60     if (node == nullLeaf) return nullLeaf;
61     // Value found at current node
62     if (value == node->value) return node;
63     // Search left subtree if value is smaller
```

ზოგადი დანიშნულება

ეს ფაილი შეიცავს RedBlackTree კლასის სრულ იმპლემენტაციას. ქვემოთ მოცემულია ფაილში არსებული ყველა ფუნქციის დეტალური აღწერა, დაჯგუფებული მათი დანიშნულების მიხედვით.

1. getSize()

- აღწერა: აბრუნებს კლასის პრივატულ ველს `nodeCount`.
- დანიშნულება: მომხმარებელს აძლევს საშუალებას გაიგოს ელემენტების რაოდენობა ხის სტრუქტურის დარღვევის გარეშე.

2. isEmpty()

- აღწერა: ამოწმებს, უდრის თუ არა `nodeCount` 0-ს.
- დანიშნულება: სწრაფი შემოწმება ხის მდგომარეობის შესახებ.

3. NullLeaf()

- აღწერა: ქმნის და აინიციალიზებს სპეციალურ "სენტინელ" კვანძს (`nullLeaf`).
- მოქმედების პრინციპი:
 - გამოყოფს მეხსიერებას `new Node()`-ით.
 - ანიჭებს ფერს: `isBlack = true` (სენტინელი ყოველთვის შავია წესების მიხედვით).
 - ანიჭებს `isLeftChild = false`.
 - მნიშვნელოვანი მომენტი: `parent`, `left` და `right` მიმთითებლებს ამისამართებს საკუთარ თავზე. ეს ქმნის ჩაკეტილ წრეს და თავიდან გვარიდებს `nullptr` გამოწვევებს (`Exceptions`) როტაციების დროს, როდესაც ვცდილობთ არარსებული შვილის მშობელზე წვდომას.

4. CopySubtree(const Node* otherNode,...)

- აღწერა: ახდენს ხის ან მისი ნაწილის ღრმა კოპირებას (`Deep Copy`).
- მოქმედება:
 - **Base Case:** თუ `otherNode` არის სენტინელი, აბრუნებს მიმდინარე ხის `nullLeaf`-ს.
 - ქმნის ახალ კვანძს (`newNode`) `otherNode`-ის მონაცემებით და ფერით.

- რეკურსიულად იძახებს CopySubtree-ს მარცხენა და მარჯვენა შვილებისთვის.
- აღადგენს isLeftChild დროშებს ახალი კვანძებისთვის.

5. ClearSubtree(Node* node)

- აღწერა: შლის გადაცემულ კვანძს და მის ყველა შთამომავალს მემსიერებიდან.
- მოქმედება: იყენებს Post-Order შემოვლას. ჯერ იძახებს თავს მარცხენა შვილზე, შემდეგ მარჯვენა შვილზე, და ბოლოს შლის (delete) თავად მშობელს. ეს თანმიმდევრობა აუცილებელია, რათა არ დავკარგოთ წვდომა შვილებზე მშობლის ნაშლის შემდეგ.

```

61     if (value < node->value)
62         return SearchRecHelper(node->left, value);
63     // Search right subtree if value is larger
64     return SearchRecHelper(node->right, value);
65 }
66
67 // Iterative helper function for searching
68 // Returns pointer to found node, or nullLeaf if not found
69 Node* RedBlackTree::SearchIterHelper(int value) const
70 {
71     Node* Walker = Root; // Start at root
72
73     // Traverse tree until nullLeaf is reached
74     while (Walker != nullLeaf)
75     {
76         if (value == Walker->value) return Walker; // Value found
77         // Go left if value is smaller, right if larger
78         Walker = (value < Walker->value) ? Walker->left : Walker->right;
79     }
80
81     return nullLeaf; // Value not found
82 }
83
84 // Public recursive search function
85 // Returns true if value exists in tree, false otherwise
86 bool RedBlackTree::SearchRec(int value) const
87 {
88     return SearchRecHelper(Root, value) != nullLeaf;
89 }
90
91 // Public iterative search function
92 // Returns true if value exists in tree, false otherwise
93 bool RedBlackTree::SearchIter(int value) const
94 {
95     return SearchIterHelper(value) != nullLeaf;
96 }
97
98 // Performs left rotation around the given node
99 void RedBlackTree::LeftRotate(Node* rotationRoot)
100 {
101     // Right child becomes new root of this subtree
102     Node* RightChild = rotationRoot->right;
103     if (RightChild == nullLeaf) return; // Cannot rotate if no right child
104
105     // Step 1: rotationRoot's right child becomes RightChild's left child
106     rotationRoot->right = RightChild->left;
107     if (RightChild->left != nullLeaf)
108     {
109         RightChild->left->parent = rotationRoot;
110         RightChild->left->isLeftChild = false; // It's now a right child
111     }
112
113     // Step 2: Update RightChild's parent
114     RightChild->parent = rotationRoot->parent;
115     // Step 3: Update parent's child pointer
116     if (rotationRoot->parent == nullLeaf)
117     {
118         // rotationRoot was the tree root
119         Root = RightChild;
120         RightChild->isLeftChild = false; // Root has no left/right designation
121     }
122 }

```

6. SearchRecHelper(Node* node, int value)

- აღწერა: ეძებს მნიშვნელობას ხეში რეკურსიული გზით.
- ლოგიკა:
 - თუ node არის nullLeaf ან მნიშვნელობა ემთხვევა, აბრუნებს node-ს.
 - თუ საძიებო მნიშვნელობა ნაკლებია, იძახებს თავს მარცხენა შვილზე.
 - თუ მეტია — მარჯვენა შვილზე.

7. SearchIterHelper(int value)

- აღწერა: ეძებს მნიშვნელობას ციკლის გამოყენებით (უფრო ეფექტურია მეხსიერების მხრივ, რადგან არ იყენებს სტეკს).
- ლოგიკა: იყენებს Walker მიმთითებელს, რომელიც მოძრაობს ფესვიდან ფოთლებისკენ while ციკლში, სანამ არ იპოვის მნიშვნელობას ან არ მიაღწევა nullLeaf-ს.

8. SearchRec და SearchIter

- აღწერა: საჯარო ფუნქციები (Wrappers), რომლებიც იძახებენ შესაბამის Helper ფუნქციებს და აბრუნებენ bool შედეგს (ნაპოვნია თუ არა).

```

121     else if (rotationRoot->isLeftChild)
122     {
123         // rotationRoot was a left child
124         rotationRoot->parent->left = RightChild;
125         RightChild->isLeftChild = true;
126     }
127     else
128     {
129         // rotationRoot was a right child
130         rotationRoot->parent->right = RightChild;
131         RightChild->isLeftChild = false;
132     }
133     // Step 4: Make rotationRoot left child of RightChild
134     RightChild->left = rotationRoot;
135     rotationRoot->parent = RightChild;
136     rotationRoot->isLeftChild = true; // rotationRoot is now left child
137 }
138
139 // Performs right rotation around the given node
140 void RedBlackTree::RightRotate(Node* rotationRoot)
141 {
142     // Left child becomes new root of this subtree
143     Node* LeftChild = rotationRoot->left;
144     if (LeftChild == nullLeaf) return; // Cannot rotate if no left child
145
146     // Step 1: rotationRoot's left child becomes LeftChild's right child
147     rotationRoot->left = LeftChild->right;
148     if (LeftChild->right != nullLeaf)
149     {
150         LeftChild->right->parent = rotationRoot;
151         LeftChild->right->isLeftChild = true; // It's now a left child
152     }
153     // Step 2: Update LeftChild's parent
154     LeftChild->parent = rotationRoot->parent;
155     // Step 3: Update parent's child pointer
156     if (rotationRoot->parent == nullLeaf)
157     {
158         // rotationRoot was the tree root
159         Root = LeftChild;
160         LeftChild->isLeftChild = false; // Root has no left/right designation
161     }
162     else if (!rotationRoot->isLeftChild)
163     {
164         // rotationRoot was a right child
165         rotationRoot->parent->right = LeftChild;
166         LeftChild->isLeftChild = false;
167     }
168     else
169     {
170         // rotationRoot was a left child
171         rotationRoot->parent->left = LeftChild;
172         LeftChild->isLeftChild = true;
173     }
174     // Step 4: Make rotationRoot right child of LeftChild
175     LeftChild->right = rotationRoot;
176     rotationRoot->parent = LeftChild;
177     rotationRoot->isLeftChild = false; // rotationRoot is now right child
178 }
179
180 // Finds the minimum value node in a subtree

```

9. LeftRotate(Node* rotationRoot)

- დანიშნულება: ასრულებს მარცხენა როტაციას მოცემული კვანძის (rotationRoot) მიმართ. გამოიყენება, როცა მარჯვენა ქვეხე არაბალანსირებულია.
- დეტალური ნაბიჯები:
 1. ინიციალიზდება RightChild (როტაციის ფესვის მარჯვენა შვილი).
 2. rotationRoot-ის მარჯვენა შვილად ხდება RightChild-ის მარცხენა შვილი.
 3. თუ ეს მარცხენა შვილი არსებობს, მისი მშობელი ხდება rotationRoot.
 4. RightChild-ის მშობელი ხდება rotationRoot-ის მშობელი (ხის სტრუქტურაში ადგილის გაცვლა).
 5. განახლდება მშობლის მიმთითებელი (იყო მარცხენა თუ მარჯვენა შვილი, შესაბამისად მიებმება RightChild).
 6. rotationRoot ხდება RightChild-ის მარცხენა შვილი.

10. RightRotate(Node* rotationRoot)

- დანიშნულება: მარცხენა როტაციის სიმეტრიული ოპერაცია. გამოიყენება მარცხენა მხარის "სიმძიმის" შესამცირებლად.
- ლოგიკა: rotationRoot-ის მარცხენა შვილი (LeftChild) იკავებს მშობლის პოზიციას, ხოლო rotationRoot ხდება მისი მარჯვენა შვილი.


```

181 // Returns pointer to node with minimum value
182 Node* RedBlackTree::Minimum(Node* node) const
183 {
184     while (node->left != nullLeaf) // Minimum is leftmost node
185         node = node->left;
186     return node;
187 }
188
189 // Replaces one subtree with another subtree
190 void RedBlackTree::Transplant(Node* oldSubtreeRoot, Node* newSubtreeRoot)
191 {
192     // Case 1: oldSubtreeRoot is the tree root
193     if (oldSubtreeRoot->parent == nullLeaf)
194     {
195         Root = newSubtreeRoot;
196         newSubtreeRoot->parent = nullLeaf;
197         newSubtreeRoot->isLeftChild = false;
198     }
199     // Case 2: oldSubtreeRoot is a left child
200     else if (oldSubtreeRoot->isLeftChild)
201     {
202         oldSubtreeRoot->parent->left = newSubtreeRoot;
203         newSubtreeRoot->parent = oldSubtreeRoot->parent;
204         newSubtreeRoot->isLeftChild = true;
205     }
206     // Case 3: oldSubtreeRoot is a right child
207     else
208     {
209         oldSubtreeRoot->parent->right = newSubtreeRoot;
210         newSubtreeRoot->parent = oldSubtreeRoot->parent;
211         newSubtreeRoot->isLeftChild = false;
212     }
213 }
214
215 // Flips the colors of a node and its children
216 // Used in Red-Black Tree rebalancing
217 void RedBlackTree::ColorFlip(Node* node)
218 {
219     if (node == nullLeaf) return;
220
221     node->isBlack = !node->isBlack; // Flip current node's color
222     // Flip left child's color if it exists
223     if (node->left != nullLeaf)
224         node->left->isBlack = !node->left->isBlack;
225     // Flip right child's color if it exists
226     if (node->right != nullLeaf)
227         node->right->isBlack = !node->right->isBlack;
228 }
229
230 // Fixes Red-Black Tree properties after insertion
231 void RedBlackTree::FixInsert(Node* insertedNode)
232 {
233     // Continue fixing while parent exists and is red (violation of property 4)
234     while (insertedNode->parent != nullLeaf && insertedNode->parent->isBlack == false)
235     {
236         Node* parent = insertedNode->parent;
237         Node* Grand = parent->parent;
238
239         // Case 1: Parent is a left child
240         if (parent->isLeftChild)

```



```

241     {
242         Node* Uncle = Grand->right;
243         // Case 1.1: Uncle is red (recoloring case)
244         if (Uncle != nullLeaf && Uncle->isBlack == false)
245         {
246             ColorFlip(Grand); // Flip grandparent's color
247             insertedNode = Grand; // Move up to grandparent
248         }
249         // Case 1.2: Uncle is black (rotation cases)
250         else
251         {
252             // Case 1.2.1: insertedNode is right child (left-right case)
253             if (insertedNode->isLeftChild == false)
254             {
255                 insertedNode = parent;
256                 LeftRotate(insertedNode); // Convert to left-left case
257                 parent = insertedNode->parent;
258                 Grand = parent->parent;
259             }
260
261             // Case 1.2.2: insertedNode is left child (left-left case)
262             parent->isBlack = true; // Parent becomes black
263             Grand->isBlack = false; // Grandparent becomes red
264             RightRotate(Grand); // Right rotate grandparent
265         }
266     }
267     // Case 2: Parent is a right child (symmetric to Case A)
268     else
269     {
270         Node* Uncle = Grand->left;
271
272         // Case 2.1: Uncle is red
273         if (Uncle->isBlack == false)
274         {
275             ColorFlip(Grand);
276             insertedNode = Grand;
277         }
278         // Case 2.2: Uncle is black
279         else
280         {
281             // Case 2.2.1: insertedNode is left child (right-left case)
282             if (insertedNode->isLeftChild == true)
283             {
284                 insertedNode = parent;
285                 RightRotate(insertedNode); // Convert to right-right case
286                 parent = insertedNode->parent;
287                 Grand = parent->parent;
288             }
289
290             // Case 2.2.2: insertedNode is right child (right-right case)
291             parent->isBlack = true; // Parent becomes black
292             Grand->isBlack = false; // Grandparent becomes red
293             LeftRotate(Grand); // Left rotate grandparent
294         }
295     }
296 }
297 // Ensure root is always black
298 if (Root != nullLeaf)
299 {
300     Root->isBlack = true;

```

```

301     Root->parent = nullLeaf;
302     Root->isLeftChild = false;
303 }
304 }
305
306 // Fixes Red-Black Tree properties after deletion
307 void RedBlackTree::FixDelete(Node* FixNode)
308 {
309     // Continue fixing while FixNode is not root and is black
310     while (FixNode != Root && FixNode->isBlack)
311     {
312         // Case 1: FixNode is a left child
313         if (FixNode->isLeftChild)
314         {
315             Node* Sibling = FixNode->parent->right;
316
317             // Case 1.1: Sibling is red
318             if (Sibling->isBlack == false)
319             {
320                 Sibling->isBlack = true;
321                 FixNode->parent->isBlack = false;
322                 LeftRotate(FixNode->parent);
323                 Sibling = FixNode->parent->right; // Update sibling after rotation
324             }
325             // Case 1.2: Both sibling's children are black
326             if (Sibling->left->isBlack && Sibling->right->isBlack)
327             {
328                 if (Sibling != nullLeaf) Sibling->isBlack = false;
329                 FixNode = FixNode->parent; // Move up the tree
330             }
331             else
332             {
333                 // Case 1.3: Sibling's right child is black (left child is red)
334                 if (Sibling->right->isBlack)
335                 {
336                     Sibling->left->isBlack = true;
337                     if (Sibling != nullLeaf) Sibling->isBlack = false;
338                     RightRotate(Sibling);
339                     Sibling = FixNode->parent->right;
340                 }
341                 // Case 1.4: Sibling's right child is red
342                 Sibling->isBlack = FixNode->parent->isBlack;
343                 FixNode->parent->isBlack = true;
344                 Sibling->right->isBlack = true;
345                 LeftRotate(FixNode->parent);
346                 FixNode = Root; // Termination case
347             }
348         }
349         // Case 2: FixNode is a right child (symmetric to Case 1)
350         else
351         {
352             Node* Sibling = FixNode->parent->left;
353
354             // Case 2.1: Sibling is red
355             if (Sibling->isBlack == false)
356             {
357                 Sibling->isBlack = true;
358                 FixNode->parent->isBlack = false;
359                 RightRotate(FixNode->parent);
360                 Sibling = FixNode->parent->left;

```

```

361 // Case 2.2: Both sibling's children are black
362 if (Sibling->left->isBlack && Sibling->right->isBlack)
363 {
364     if (Sibling != nullLeaf) Sibling->isBlack = false;
365     FixNode = FixNode->parent;
366 }
367 else
368 {
369     // Case 2.3: Sibling's left child is black (right child is red)
370     if (Sibling->left->isBlack)
371     {
372         Sibling->right->isBlack = true;
373         if (Sibling != nullLeaf) Sibling->isBlack = false;
374         LeftRotate(Sibling);
375         Sibling = FixNode->parent->left;
376     }
377     // Case 2.4: Sibling's left child is red
378     Sibling->isBlack = FixNode->parent->isBlack;
379     FixNode->parent->isBlack = true;
380     Sibling->left->isBlack = true;
381     RightRotate(FixNode->parent);
382     FixNode = Root; // Termination case
383 }
384 }
385 }
386 }
387
388 // Ensure FixNode is black (property 1)
389 FixNode->isBlack = true;
390 }
391
392 // Default constructor - creates empty Red-Black Tree
393 RedBlackTree::RedBlackTree() : Root(nullptr), nullLeaf(nullptr), nodeCount(0)
394 {
395     NullLeaf(); // Create sentinel node
396     Root = nullLeaf; // Empty tree root points to sentinel
397 }
398
399 // Copy constructor - creates deep copy of another tree
400 RedBlackTree::RedBlackTree(const RedBlackTree& other)
401 : Root(nullptr), nullLeaf(nullptr), nodeCount(other.nodeCount)
402 {
403     NullLeaf(); // Create our own sentinel
404     Root = nullLeaf; // Initialize to empty
405
406     // Copy only if source tree is not empty
407     if (other.Root != other.nullLeaf)
408     {
409         // Recursively copy entire tree
410         Root = CopySubtree(other.Root, other.nullLeaf, nullLeaf);
411         Root->parent = nullLeaf;
412         Root->isLeftChild = false;
413         Root->isBlack = true; // Root must be black
414     }
415 }
416
417 // Assignment operator - makes deep copy of another tree
418 // Returns reference to this tree
419 RedBlackTree& RedBlackTree::operator = (const RedBlackTree& other)
420 {

```

11. ColorFlip(Node* node)

- აღწერა: ცვლის კვანძისა და მისი ორივე შვილის ფერს საპირისპიროთი.
- გამოყენება: ძირითადად FixInsert-ის დროს, როდესაც "ბიძა" წითელია.

12. FixInsert(Node* insertedNode)

- დანიშნულება: აღადგენს ხის თვისებებს ახალი წითელი კვანძის ჩამატების შემდეგ.
- ალგორითმი: ციკლი გრძელდება, სანამ მშობელი წითელია.
 - Case 1 (ბიძა წითელია): ხდება ColorFlip ბაბუაზე (Grandparent). ბაბუა წითლდება, მშობელი და ბიძა შავდებიან. შემონმება გრძელდება ბაბუაზე.
 - Case 2 (ბიძა შავია, ზიგ-ზაგი): თუ კვანძი მშობლის საპირისპირო მხარესაა (მაგ. მშობელი მარცხენაა, კვანძი მარჯვენა), კეთდება როტაცია მშობლის მიმართ, რათა მივიღოთ სწორხაზოვანი განლაგება.
 - Case 3 (ბიძა შავია, ხაზი): მშობელი შავდება, ბაბუა წითლდება და კეთდება როტაცია პაპის მიმართ. ეს ასრულებს ბალანსირებას.
 - ბოლოს: ფესვი (Root) ყოველთვის შავად იღებება.

13. FixDelete(Node* FixNode)

- დანიშნულება: ყველაზე რთული ფუნქცია. აღადგენს ბალანსირებულ სტრუქტურას შავი კვანძის წაშლის შემდეგ.
- ალგორითმი: იყენებს Sibling (ძმა) კვანძს.
 - Case 1: ძმა წითელია -> როტაცია მშობელზე, ძმა ხდება შავი.
 - Case 2: ძმა და მისი შვილები შავია -> ძმა წითლდება, "Double Black" ადის მშობელზე.
 - Case 3: ძმა შავია, ახლო შვილი წითელია -> როტაცია ძმაზე.
 - Case 4: ძმა შავია, შორეული შვილი წითელია -> როტაცია მშობელზე, ფერების გაცვლა. ეს საბოლოოდ აგვარებს პრობლემას.

14. Transplant(Node* old, Node* new)

- აღწერა: დამხმარე ფუნქცია, რომელიც ანაცვლებს ერთ ქვეხეს მეორეთი. ის მხოლოდ მშობლების მიმთითებლებს ანახლებს და არ ზრუნავს შვილების ბმულებზე.

15. Minimum(Node* node)

- აღწერა: პოულობს მინიმალურ მნიშვნელობას ქვეხეში (სულ მარცხნივ სიარულით). გამოიყენება წაშლისას მემკვიდრის (Successor) საპოვნელად.

16. RedBlackTree() (Default)

- ქმნის ცარიელ ხეს: იძახებს NullLeaf-ს და Root-ს უტოლებს მას.

17. RedBlackTree(const RedBlackTree& other) (Copy)

- ქმნის ახალ სენტინელს და იძახებს CopySubtree-ს სხვა ხის ფესვიდან.


```

421 // Check for self-assignment
422 if (this == &other) return *this;
423 // Clear current tree
424 ClearSubtree(Root);
425 Root = nullLeaf;
426 // Copy node count
427 nodeCount = other.nodeCount;
428 // Copy tree structure if source is not empty
429 if (other.Root != other.nullLeaf)
430 {
431     Root = CopySubtree(other.Root, other.nullLeaf, nullLeaf);
432     Root->parent = nullLeaf;
433     Root->isLeftChild = false;
434     Root->isBlack = true; // Root must be black
435 }
436
437 return *this;
438 }
439
440 // Destructor - cleans up all dynamically allocated memory
441 RedBlackTree::~RedBlackTree()
442 {
443     ClearSubtree(Root); // Delete all tree nodes
444     delete nullLeaf; // Delete sentinel node
445     Root = nullLeaf = nullptr; // Prevent dangling pointers
446 }
447
448 // Recursive helper for BST insertion
449 // Returns root of subtree after insertion
450 Node* RedBlackTree::InsertBST(Node* currentRoot, Node* parent, int value, Node*& insertedNode, bool isLeftChild)
451 {
452     // Found insertion point
453     if (currentRoot == nullLeaf)
454     {
455         // Create new red node
456         insertedNode = new Node(value, false, isLeftChild, parent, nullLeaf, nullLeaf);
457         return insertedNode;
458     }
459     // Value already exists - duplicate not allowed
460     if (value == currentRoot->value)
461     {
462         insertedNode = nullLeaf; // Signal duplicate
463         return currentRoot;
464     }
465     // Insert in left subtree
466     if (value < currentRoot->value)
467         currentRoot->left = InsertBST(currentRoot->left, currentRoot, value, insertedNode, true);
468     // Insert in right subtree
469     else
470         currentRoot->right = InsertBST(currentRoot->right, currentRoot, value, insertedNode, false);
471
472     return currentRoot;
473 }
474
475 // Iterative insertion of a value into Red-Black Tree
476 void RedBlackTree::InsertIter(int value)
477 {
478     Node* parent = nullLeaf;
479     Node* Walker = Root;
480     bool GoLeft = false; // Direction to go from parent

```



```

481
482 // Find insertion point
483 while (Walker != nullLeaf)
484 {
485     parent = Walker;
486     // Duplicate value - do nothing
487     if (value == Walker->value) return;
488     // Go left if value is smaller
489     if (value < Walker->value)
490     {
491         Walker = Walker->left;
492         GoLeft = true;
493     }
494     // Go right if value is larger
495     else
496     {
497         Walker = Walker->right;
498         GoLeft = false;
499     }
500 }
501 // Create new red node
502 Node* newNode = new Node(value, false, (parent == nullLeaf ? false : GoLeft),
503     parent, nullLeaf, nullLeaf);
504
505 // Case 1: Tree was empty - new node becomes root
506 if (parent == nullLeaf)
507 {
508     Root = newNode;
509     Root->parent = nullLeaf;
510     Root->isLeftChild = false;
511     Root->isBlack = true; // Root must be black
512     ++nodeCount;
513     return;
514 }
515 // Case 2: Insert as left or right child of parent
516 if (GoLeft)
517     parent->left = newNode;
518 else
519     parent->right = newNode;
520
521 // Fix Red-Black Tree properties
522 FixInsert(newNode);
523 ++nodeCount;
524 }
525
526 // Recursive insertion of a value into Red-Black Tree
527 void RedBlackTree::InsertRec(int value)
528 {
529     Node* insertedNode = nullLeaf;
530
531     // Perform BST insertion (ignoring colors)
532     Root = InsertBST(Root, nullLeaf, value, insertedNode, false);
533     // Update root properties
534     if (Root != nullLeaf)
535     {
536         Root->parent = nullLeaf;
537         Root->isLeftChild = false;
538     }
539     // If insertion was successful (not duplicate)
540     if (insertedNode != nullLeaf) {

```

```

541         FixInsert(insertedNode); // Fix Red-Black properties
542         ++nodeCount;
543     }
544     // If duplicate, ensure root is black
545     else if (Root != nullLeaf)
546         Root->isBlack = true;
547 }
548
549 // Internal function to delete a node given its pointer
550 void RedBlackTree::RemoveByNode(Node* Target)
551 {
552     if (Target == nullLeaf) return; // Nothing to delete
553
554     Node* RemovedNode = Target; // Node being removed
555     bool wasBlack = RemovedNode->isBlack; // Color affects fixup
556     Node* FixNode = nullLeaf; // Node to start fixup from
557
558     // Case 1: Target has no left child
559     if (Target->left == nullLeaf)
560     {
561         FixNode = Target->right;
562         Transplant(Target, Target->right);
563     }
564     // Case 2: Target has no right child
565     else if (Target->right == nullLeaf)
566     {
567         FixNode = Target->left;
568         Transplant(Target, Target->left);
569     }
570     // Case 3: Target has two children
571     else
572     {
573         // Find successor (minimum in right subtree)
574         RemovedNode = Minimum(Target->right);
575         wasBlack = RemovedNode->isBlack;
576         FixNode = RemovedNode->right;
577         // Special case: successor is Target's immediate right child
578         if (RemovedNode->parent == Target)
579         {
580             FixNode->parent = RemovedNode;
581             FixNode->isLeftChild = false;
582         }
583         // General case: successor is deeper in right subtree
584         else
585         {
586             Transplant(RemovedNode, RemovedNode->right);
587             RemovedNode->right = Target->right;
588             RemovedNode->right->parent = RemovedNode;
589             RemovedNode->right->isLeftChild = false;
590         }
591         // Replace Target with its successor
592         Transplant(Target, RemovedNode);
593         RemovedNode->left = Target->left;
594         RemovedNode->left->parent = RemovedNode;
595         RemovedNode->left->isLeftChild = true;
596         // Copy color from Target
597         RemovedNode->isBlack = Target->isBlack;
598         RemovedNode->isLeftChild = Target->isLeftChild;
599     }
600 }

```

18. operator= (Assignment)

- ასუფთავებს მიმდინარე ხეს (ClearSubtree), და შემდეგ აკოპირებს ახალს.

19. ~RedBlackTree() (Destructor)

- იძახებს **ClearSubtree(Root)**-ს და ბოლოს შლის **nullLeaf**-ს, რათა არ მოხდეს მეხსიერების გაჟონვა (**Memory Leak**).

20. **InsertBST(...)**

- აღწერა: რეკურსიული დამხმარე ფუნქცია, რომელიც პოულობს ჩასამატებელ ადგილს, ქმნის ახალ კვანძს და აბმამს მას მშობელს. არ აკეთებს ბალანსირებას.

21. **InsertIter(int value)**

- აღწერა: იტერაციული ჩამატება.
- ლოგიკა: **while** ციკლით ჩადის ფოთლამდე, ქმნის ახალ წითელ კვანძს, აბმამს მშობელს და იძახებს **FixInsert**-ს.

22. **InsertRec(int value)**

- აღწერა: რეკურსიული ჩამატება. იძახებს **InsertBST**-ს და შემდეგ **FixInsert**-ს.

```

601 // Delete the target node
602 delete Target;
603 // If removed node was black, need to fix tree properties
604 if (wasBlack)
605     FixDelete(FixNode);
606 // Ensure root is black and properly set
607 if (Root != nullLeaf)
608 {
609     Root->isBlack = true;
610     Root->parent = nullLeaf;
611     Root->isLeftChild = false;
612 }
613 --nodeCount; // Decrement node count
614 }
615
616 // Iterative removal of a value from tree
617 void RedBlackTree::RemoveIter(int value)
618 {
619     Node* Target = SearchIterHelper(value); // Find node to delete
620     RemoveByNode(Target); // Delete it
621 }
622
623 // Recursive removal of a value from tree
624 void RedBlackTree::RemoveRec(int value)
625 {
626     Node* Target = SearchRecHelper(Root, value); // Find node to delete
627     RemoveByNode(Target); // Delete it
628 }
629
630 // Public in-order traversal interface
631 void RedBlackTree::DisplayInOrder(std::ostream& out) const
632 {
633     DisplayInOrder(out, Root);
634     out << "\n";
635 }
636
637 // Public pre-order traversal interface
638 void RedBlackTree::DisplayPreOrder(std::ostream& out) const
639 {
640     DisplayPreOrder(out, Root);
641     out << "\n";
642 }
643
644 // Public post-order traversal interface
645 void RedBlackTree::DisplayPostOrder(std::ostream& out) const
646 {
647     DisplayPostOrder(out, Root);
648     out << "\n";
649 }
650
651 // Recursive in-order traversal helper
652 void RedBlackTree::DisplayInOrder(std::ostream& out, Node* node) const
653 {
654     if (node == nullLeaf) return; // Base case
655
656     // In-order: left, node, right
657     DisplayInOrder(out, node->left);
658     out << node->value << "(" << (node->isBlack ? "B" : "R") << ") ";
659     DisplayInOrder(out, node->right);
660 }

```



```

661
662 // Recursive pre-order traversal helper
663 void RedBlackTree::DisplayPreOrder(std::ostream& out, Node* node) const
664 {
665     if (node == nullLeaf) return; // Base case
666
667     // Pre-order: node, left, right
668     out << node->value << "(" << (node->isBlack ? "B" : "R") << ") ";
669     DisplayPreOrder(out, node->left);
670     DisplayPreOrder(out, node->right);
671 }
672
673 // Recursive post-order traversal helper
674 void RedBlackTree::DisplayPostOrder(std::ostream& out, Node* node) const
675 {
676     if (node == nullLeaf) return; // Base case
677
678     // Post-order: left, right, node
679     DisplayPostOrder(out, node->left);
680     DisplayPostOrder(out, node->right);
681     out << node->value << "(" << (node->isBlack ? "B" : "R") << ") ";
682 }

```

23. RemoveByNode(Node* Target)

- აღწერა: შლის კონკრეტულ კვანძს.
- ლოგიკა:
 - თუ კვანძს 0 ან 1 შვილი ჰყავს, იყენებს Transplant-ს ჩასანაცვლებლად.
 - თუ 2 შვილი ჰყავს: პოულობს მემკვიდრეს (Minimum მარჯვენა მხარეს), გადააქვს მისი მონაცემი და შლის მემკვიდრეს.
 - თუ ფიზიკურად წაშლილი კვანძი შავი იყო, იძახებს FixDelete-ს.

24. RemoveIter და RemoveRec

- აღწერა: საჯარო ფუნქციები. ჯერ პოულობენ კვანძს (Search ფუნქციებით) და შემდეგ იძახებენ RemoveByNode-ს.

25. DisplayInOrder (და მისი Helper)

- ლოგიკა: მარცხენა -> მშობელი -> მარჯვენა. ბეჭდავს ელემენტებს ზრდადობის მიხედვით.

26. DisplayPreOrder (და მისი Helper)

- ლოგიკა: მშობელი -> მარცხენა -> მარჯვენა. გამოსადეგია ხის სტრუქტურის აღსადგენად/კოპირებისთვის.

27. DisplayPostOrder (და მისი Helper)

- ლოგიკა: მარცხენა -> მარჯვენა -> მშობელი. გამოიყენება ClearSubtree-ში ნაშლისთვის.

მედი 4 — Driver.cpp

```

1  ✓ #include <iostream>
2  | #include "RedBlackTree.h"
3
4  ✓ int main()
5  {
6      std::cout << "===== Iterative Test =====";
7      // Test 1: Iterative operations
8      RedBlackTree iterTree;
9      std::cout << "\nIs Tree Empty? " << (iterTree.isEmpty() ? "Yes" : "No")
10         << "\nSize: " << iterTree.getSize() << "\n";
11      // Insert values using iterative method
12      iterTree.InsertIter(10);
13      iterTree.InsertIter(13);
14      iterTree.InsertIter(14);
15      iterTree.InsertIter(15);
16      iterTree.InsertIter(11);
17      iterTree.InsertIter(9);
18      iterTree.InsertIter(12);
19      iterTree.InsertIter(10); // Duplicate - should be ignored
20
21      std::cout << "\n\tDisplay of the Tree After Insertion\n";
22      std::cout << "Inorder: ";
23      iterTree.DisplayInOrder(); // Displays in sorted order
24      std::cout << "Preorder: ";
25      iterTree.DisplayPreOrder(); // Displays root first
26      std::cout << "Postorder: ";
27      iterTree.DisplayPostOrder(); // Displays leaves first
28      std::cout << "New Size: " << iterTree.getSize() << "\n\n";
29
30      // Search tests
31      std::cout << "Searching For 11: " << (iterTree.SearchIter(11) ? "FOUND\n" : "NOT FOUND\n");
32      std::cout << "Searching For 77: " << (iterTree.SearchIter(77) ? "FOUND\n" : "NOT FOUND\n");
33
34      // Delete tests
35      iterTree.RemoveIter(13);
36      iterTree.RemoveIter(10);
37      iterTree.RemoveIter(100); // Non-existent value
38
39      std::cout << "\n\tDisplay After Removing\n";
40      std::cout << "Inorder: ";
41      iterTree.DisplayInOrder();
42      std::cout << "Size After Remove: " << iterTree.getSize() << "\n\n";
43
44      // Test 2: Recursive operations
45      std::cout << "===== Recursive Test =====";
46      RedBlackTree recTree;
47      std::cout << "\nIs Tree Empty? " << (recTree.isEmpty() ? "Yes" : "No")
48         << "\nSize: " << recTree.getSize() << "\n";
49
50      // Insert values using recursive method
51      recTree.InsertRec(12);
52      recTree.InsertRec(18);
53      recTree.InsertRec(21);
54      recTree.InsertRec(34);
55      recTree.InsertRec(59);

```

```

55     recTree.InsertRec(59);
56     recTree.InsertRec(72);
57     recTree.InsertRec(3);
58     recTree.InsertRec(21); // Duplicate - should be ignored
59
60     std::cout << "\n\tDisplay of the Tree After Insertion\n";
61     std::cout << "Inorder: ";
62     recTree.DisplayInOrder();
63     std::cout << "Preorder: ";
64     recTree.DisplayPreOrder();
65     std::cout << "Postorder: ";
66     recTree.DisplayPostOrder();
67     std::cout << "New Size: " << recTree.getSize() << "\n\n";
68
69     // Search tests
70     std::cout << "Searching For 12: " << (recTree.SearchRec(12) ? "FOUND\n" : "NOT FOUND\n");
71     std::cout << "Search For 57: " << (recTree.SearchRec(57) ? "FOUND\n" : "NOT FOUND\n");
72
73     // Delete tests
74     recTree.RemoveRec(13);
75     recTree.RemoveRec(10);
76     recTree.RemoveRec(77); // Non-existent values
77
78     std::cout << "\n\tDisplay After Removing\n";
79     std::cout << "Inorder: ";
80     recTree.DisplayInOrder();
81     std::cout << "Size After Remove: " << recTree.getSize() << "\n\n";
82
83     // Test 3: Copy constructor
84     std::cout << "===== Copy Constructor Test =====";
85     RedBlackTree Copied(recTree); // Create copy using copy constructor
86
87     std::cout << "\n\nDisplay By Preorder: ";
88     Copied.DisplayPreOrder();
89     std::cout << "Size: " << Copied.getSize() << "\n\n";
90
91     // Test 4: Assignment operator
92     std::cout << "===== Assignment Operator Test =====";
93     RedBlackTree RBT;
94     RBT = recTree; // Create copy using assignment operator
95
96     std::cout << "\n\nDisplay By Preorder: ";
97     RBT.DisplayPreOrder();
98     std::cout << "Size: " << RBT.getSize() << "\n";
99 }

```

ზოგადი დანიშნულება

Driver.cpp არის დრაივერი პროგრამა (Driver Code), რომლის მიზანია **RedBlackTree** კლასის ფუნქციონალის დემონსტრირება და ვალიდაცია.

ტესტირების შედეგები

1. იტერაციული ტესტი:

- ხეში ემატება რიცხვები: 10, 13, 14, 15, 11, 9, 12.
- სპეციალურად შერჩეული თანმიმდევრობაა, რომელიც ინვერს მრავალგზის როტაციას და ფერების ცვლილებას.
- მოწმდება დუბლიკატის იგნორირება.
- სრულდება ძებნა და ნაშლა იტერაციული მეთოდებით.

```
===== Iterative Test =====
Is Tree Empty? Yes
Size: 0

      Display of the Tree After Insertion
Inorder: 9(B) 10(R) 11(B) 12(R) 13(B) 14(B) 15(R)
Preorder: 13(B) 10(R) 9(B) 11(B) 12(R) 14(B) 15(R)
Postorder: 9(B) 12(R) 11(B) 10(R) 15(R) 14(B) 13(B)
New Size: 7

Searching For 11: FOUND
Searching For 77: NOT FOUND

      Display After Removing
Inorder: 9(B) 11(R) 12(B) 14(B) 15(B)
Size After Remove: 5
```


2. რეკურსიული ტესტი:

- მოწმდება InsertRec, SearchRec და RemoveRec ფუნქციები.
- დასტურდება, რომ რეკურსიული იმპლემენტაცია იძლევა იდენტურ შედეგს.

```
===== Recursive Test =====  
Is Tree Empty? Yes  
Size: 0  
  
      Display of the Tree After Insertion  
Inorder: 3(R) 12(B) 18(B) 21(B) 34(R) 59(B) 72(R)  
Preorder: 18(B) 12(B) 3(R) 34(R) 21(B) 59(B) 72(R)  
Postorder: 3(R) 12(B) 21(B) 72(R) 59(B) 34(R) 18(B)  
New Size: 7  
  
Searching For 12: FOUND  
Search For 57: NOT FOUND  
  
      Display After Removing  
Inorder: 3(R) 12(B) 18(B) 21(B) 34(R) 59(B) 72(R)  
Size After Remove: 7
```

3. კოპირებისა და მინიჭების ტესტი:

- იქმნება ხის ასლი კონსტრუქტორით (Copied).
- იქმნება ხის ასლი მინიჭების ოპერატორით (RBT = recTree).
- ეკრანზე გამოტანილი შედეგები ადასტურებს, რომ სტრუქტურა სრულად და სწორად გადავიდა ახალ ობიექტებში. დასასრული

```
===== Copy Constructor Test =====
```

```
Display By Preorder: 18(B) 12(B) 3(R) 34(R) 21(B) 59(B) 72(R)  
Size: 7
```

```
===== Assigment Operator Test =====
```

```
Display By Preorder: 18(B) 12(B) 3(R) 34(R) 21(B) 59(B) 72(R)  
Size: 7
```